

EC4010-DIGITAL DESIGN
ASSIGNMENT-02
ALU

BANDARA H.G.T.D.

2022/E/048

EC4010

SEMESTER 04

2025/02/07

➤ **Question 01:** Define the Arithmetic Logic Unit (ALU)

The **Arithmetic Logic Unit (ALU)** is a fundamental part of a computer's central processing unit (CPU) responsible for performing arithmetic operations (like addition, subtraction, multiplication, and division) and logical operations (such as AND, OR, NOT, and XOR). It acts as the brain's calculator, executing the mathematical and decision-making tasks that allow a computer to process data efficiently.

➤ **Question 2:** List and Explain the Key Operations Performed by an ALU

The **Arithmetic Logic Unit (ALU)** performs two main types of operations, **arithmetic operations** and **logical operations**. Here's a breakdown of its key functions:

1. Arithmetic Operations

- **Addition (+):** Adds two numbers together.
- **Subtraction (-):** Subtracts one number from another.
- **Multiplication (×):** Multiplies two numbers (can be done using repeated addition or specialized circuits).
- **Division (÷):** Divides one number by another (often using repeated subtraction or shift operations).
- **Increment & Decrement:** Increases or decreases a value by 1.

2. Logical Operations

- **AND:** Outputs 1 if both inputs are 1, otherwise outputs 0.
- **OR:** Outputs 1 if at least one input is 1.
- **NOT:** Inverts the input (0 becomes 1, and 1 becomes 0).
- **XOR (Exclusive OR):** Outputs 1 if inputs are different; otherwise, outputs 0.

3. Bitwise Operations

- **Bitwise AND, OR, XOR, NOT:** Perform logical operations at the bit level.
- **Shift Left (<<):** Moves bits to the left, multiplying the number by 2.
- **Shift Right (>>):** Moves bits to the right, dividing the number by 2.

4. Comparison Operations

- **Equal to (==):** Checks if two values are equal.

- **Greater than (>):** Checks if one value is larger than another.
- **Less than (<):** Checks if one value is smaller than another.

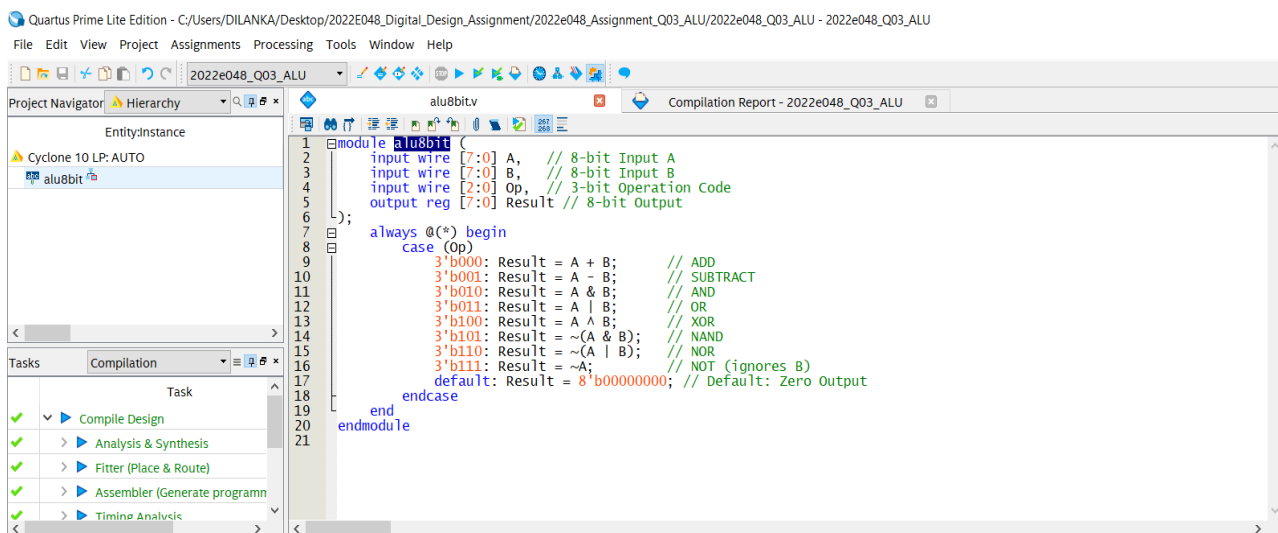
5. Boolean Operations

- Used in decision-making and control flow (e.g., determining if a condition is true or false in programming).

The ALU is the computational powerhouse of a CPU, enabling a computer to process data efficiently and perform logical decisions in real-time.

➤ Question 3: Design a Simple ALU

I am utilizing Intel Quartus software for designing the ALU.



```

1 module alu8bit (
2     input wire [7:0] A, // 8-bit Input A
3     input wire [7:0] B, // 8-bit Input B
4     input wire [2:0] Op, // 3-bit Operation Code
5     output reg [7:0] Result // 8-bit Output
6 );
7
8 always @(*) begin
9     case (Op)
10        3'b000: Result = A + B; // ADD
11        3'b001: Result = A - B; // SUBTRACT
12        3'b010: Result = A & B; // AND
13        3'b011: Result = A | B; // OR
14        3'b100: Result = A ^ B; // XOR
15        3'b101: Result = ~(A & B); // NAND
16        3'b110: Result = ~(A | B); // NOR
17        3'b111: Result = ~A; // NOT (ignores B)
18        default: Result = 8'b00000000; // Default: Zero Output
19    endcase
20 end
21 endmodule
  
```

FIGURE 01: VERILOG CODE FOR SIMPLE ALU

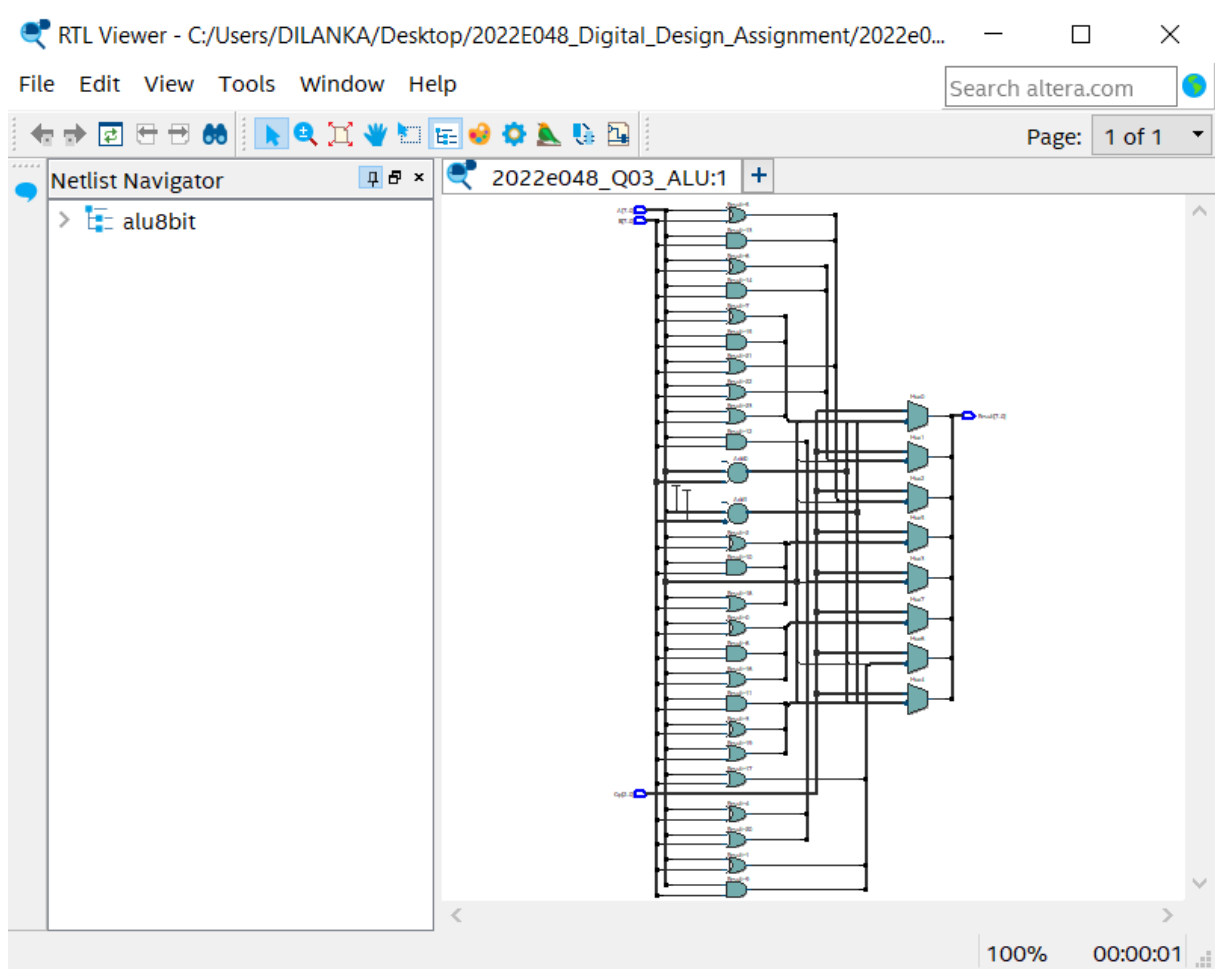


FIGURE 02: CIRCUIT DIAGRAM FOR SIMPLE ALU

```

1 timescale 1ns/1ps;
2 module tb_alu8bit();
3   reg [7:0] A, B;
4   reg [2:0] Op;
5   wire [7:0] Result;
6
7   // Instantiate the ALU
8   alu8bit uut (
9     .A(A),
10    .B(B),
11    .Op(Op),
12    .Result(Result)
13  );
14
15  initial begin
16    A = 8'b00001111; B = 8'b00000011;
17
18    Op = 3'b000; #10; // ADD
19    Op = 3'b001; #10; // SUBTRACT
20    Op = 3'b010; #10; // AND
21    Op = 3'b011; #10; // OR
22    Op = 3'b100; #10; // XOR
23    Op = 3'b101; #10; // NAND
24    Op = 3'b110; #10; // NOR
25    Op = 3'b111; #10; // NOT (ignores B)
26
27    $stop; // Stop the simulation
28  end
29 endmodule
30

```

FIGURE 03: TEST BENCH CODE FOR SIMPLE ALU

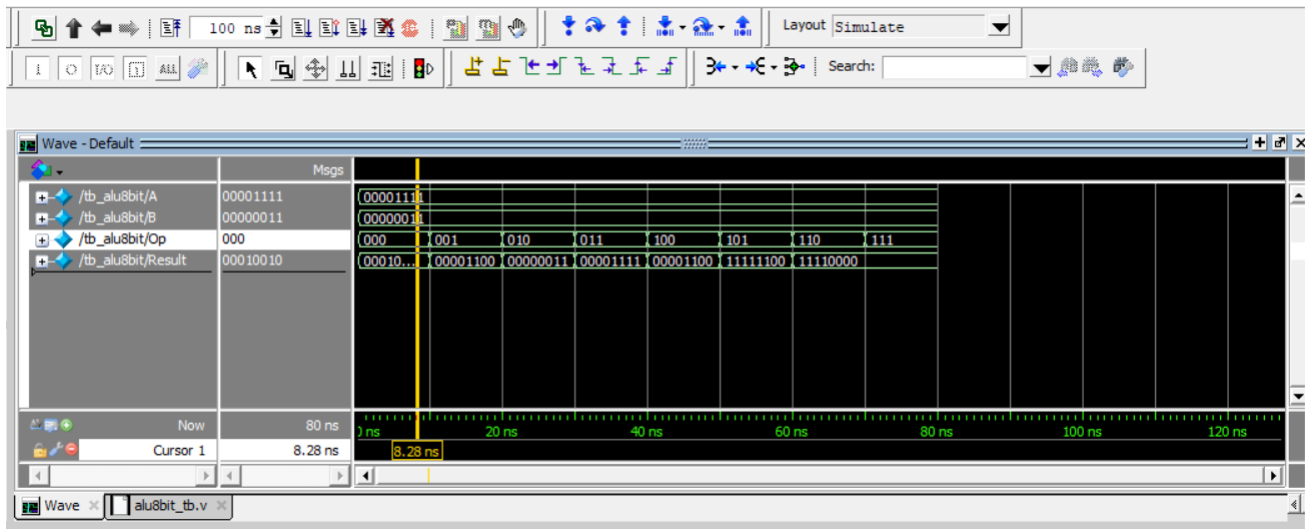


FIGURE 04: TEST BENCH OUTPUT FOR SIMPLE ALU

➤ **Question 4:** Use Cases for ALU Operations

The **Arithmetic Logic Unit (ALU)** is used in various computing tasks that require arithmetic and logical processing. Here are some key **use cases** for ALU operations:

1. Computational Tasks

- Performing mathematical calculations in **scientific applications** and **engineering simulations**.
- Handling **floating-point and integer arithmetic** in complex computations.

2. Data Processing

- Manipulating and transforming data in **databases** and **spreadsheets**.
- Encoding and decoding data in **cryptography and compression algorithms**.

3. Decision Making in Programs

- Executing conditional statements (if-else, switch-case) in programming based on **comparison operations**.

- Supporting logical decisions in **artificial intelligence** and **machine learning models**.

4. Signal Processing

- Performing **Fast Fourier Transform (FFT)** and **digital filtering** in **audio and video processing**.
- Processing signals in **communication systems**, like modulating and demodulating data.

5. Control and Embedded Systems

- Running computations in **microcontrollers** for automation and IoT devices.
- Handling arithmetic in **robotics** for sensor data processing and motor control.

6. Cryptography and Security

- Executing bitwise operations for **encryption and decryption**.
- Performing logical operations for **hashing algorithms** used in cybersecurity.

7. Operating System Functions

- Managing **memory addresses** for efficient data storage and retrieval.
- Controlling system tasks like **scheduling and resource allocation**.

The ALU is the heart of computation in modern processors, enabling everything from basic calculations to advanced machine intelligence.

REFERENCES:

- ❖ Stallings, W. (2020). *Computer Organization and Architecture: Designing for Performance* (11th ed.). Pearson.
- ❖ Mano, M. M., & Kime, C. R. (2017). *Logic and Computer Design Fundamentals* (5th ed.). Pearson.
- ❖ Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Elsevier.
- ❖ Brown, S., & Vranesic, Z. (2013). *Fundamentals of Digital Logic with Verilog Design* (3rd ed.). McGraw-Hill.